

CSC4043

...

Parallel Processing Lab Session II

A.D.Dulani S De Silva

Recap

Write a program for getting the thread count, with the thread ID

Recap

Write a program for getting the thread count, with the thread ID

```
#include <omp.h>
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char* argv[])
{
    #pragma omp parallel
    {
        int numt = omp_get_num_threads();    // Get the total number of OMP threads

        int tid = omp_get_thread_num();      // Get the thread id of this thread
        // print "Hello world" along with thread id and number of threads
        printf("Hello World from thread %d of %d.\n", tid, numt);
    }
    return 0;
}
```

Content

- Data Scoping
- Synchronization - coordinating thread execution ensuring data consistency.
- Work Distribution - Division of work among multiple threads.

Data Scoping

OpenMP is based upon the shared memory programming model, so most variables are shared by default.

Types of variables:

1. Shared Variables - all threads use the same variable value.
2. Private Variables - each thread gets its own separate copy.

OpenMP Data Scope Clauses. - How to control behaviour of variables

1. PRIVATE → each thread gets its own copy.
2. FIRSTPRIVATE → private copy initialized with original value.
3. LASTPRIVATE → last thread's value is copied back after execution.
4. SHARED → all threads use the same variable.

Shared

- All the threads read/write the same variable.
- Default for most vars
- Needs protection if written.

Without atomic counter++ is a race condition, read increment write can interleave

Output:

Final counter value: 8

```
#include <omp.h>
#include <stdio.h>
```

```
// Variable is shared among all threads
```

```
int main() {
    int shared_counter = 0;

    #pragma omp parallel shared(shared_counter)
    {
        #pragma omp atomic
        shared_counter++;

        #pragma omp barrier
        if(omp_get_thread_num() == 0) {
            printf("Final counter value: %d\n", shared_counter);
        }
    }

    return 0;
}
```

private

- Each thread gets its own uninitialized copy.
- Changes do not affect the original.

I after the parallel region is still 100. The original is untouched.

```
#include <omp.h>
#include <stdio.h>

// Variables are private to each thread

int main() {
    int i = 100;

    #pragma omp parallel private(i)
    {
        i = omp_get_thread_num(); // Each thread gets its own copy
        printf("Thread %d has i = %d\n", omp_get_thread_num(), i);
    }

    printf("After parallel region, i = %d\n", i); // Original value preserved
    return 0;
}
```

private

Thread 1 has i = 1

Thread 5 has i = 5

Thread 2 has i = 2

Thread 3 has i = 3

Thread 0 has i = 0

Thread 6 has i = 6

Thread 4 has i = 4

Thread 7 has i = 7

After parallel region, i = 100

Thread 1 has i = 1

Thread 3 has i = 3

Thread 2 has i = 2

Thread 0 has i = 0

Thread 6 has i = 6

Thread 7 has i = 7

Thread 4 has i = 4

Thread 5 has i = 5

After parallel region, i = 100

firstprivate

- Private copy initialized from original value before the region starts.

Every thread starts with $x = 5$, then they diverge.

```
#include <omp.h>
#include <stdio.h>

// Initializes private copies with original value

int main() {
    int x = 5;

    #pragma omp parallel firstprivate(x)
    {
        printf("Thread %d starts with x = %d\n", omp_get_thread_num(),
x);
        x += omp_get_thread_num();
        printf("Thread %d ends with x = %d\n", omp_get_thread_num(),
x);
    }

    printf("Original x = %d\n", x); // Still 5
    return 0;
}
```

firstprivate

Thread 6 starts with $x = 5$
Thread 6 ends with $x = 11$
Thread 0 starts with $x = 5$
Thread 0 ends with $x = 5$
Thread 5 starts with $x = 5$
Thread 5 ends with $x = 10$
Thread 7 starts with $x = 5$
Thread 7 ends with $x = 12$
Thread 3 starts with $x = 5$
Thread 3 ends with $x = 8$
Thread 2 starts with $x = 5$
Thread 2 ends with $x = 7$
Thread 1 starts with $x = 5$
Thread 1 ends with $x = 6$
Thread 4 starts with $x = 5$
Thread 4 ends with $x = 9$
Original $x = 5$

Thread 1 starts with $x = 5$
Thread 1 ends with $x = 6$
Thread 0 starts with $x = 5$
Thread 0 ends with $x = 5$
Thread 4 starts with $x = 5$
Thread 4 ends with $x = 9$
Thread 6 starts with $x = 5$
Thread 6 ends with $x = 11$
Thread 5 starts with $x = 5$
Thread 5 ends with $x = 10$
Thread 2 starts with $x = 5$
Thread 2 ends with $x = 7$
Thread 3 starts with $x = 5$
Thread 3 ends with $x = 8$
Thread 7 starts with $x = 5$
Thread 7 ends with $x = 12$
Original $x = 5$

lastprivate

- Private copy, but the value from the Last loop iteration is copied back.

Final $x = 8$ because the last loop iteration ($i=4$) sets $x = 4 * 2 = 8$

```
#include <omp.h>
#include <stdio.h>

// Final value is copied back to original variable

int main() {
    int i, x;

    #pragma omp parallel for lastprivate(x)
    for(i = 0; i < 5; i++) {
        x = i * 2;
        printf("Thread %d sets x to %d\n", omp_get_thread_num(), x);
    }

    printf("Final value of x = %d\n", x); // Will be 8 (last iteration)
    return 0;
}
```

lastprivate

Thread 1 sets x to 2

Thread 3 sets x to 6

Thread 0 sets x to 0

Thread 2 sets x to 4

Thread 4 sets x to 8

Final value of x = 8

Thread 2 sets x to 4

Thread 1 sets x to 2

Thread 4 sets x to 8

Thread 0 sets x to 0

Thread 3 sets x to 6

Final value of x = 8

threadprivate

- Each thread keeps its own copy across multiple parallel regions.

Values survive across two separate parallel regions for the same thread.

```
#include <omp.h>
#include <stdio.h>
```

```
// Global variables private to each thread
```

```
int thread_private_var;
#pragma omp threadprivate(thread_private_var)
```

```
int main() {
    #pragma omp parallel
    {
        thread_private_var = omp_get_thread_num();
        printf("First parallel region: Thread %d has value %d\n",
            omp_get_thread_num(), thread_private_var);
    }

    #pragma omp parallel
    {
        printf("Second parallel region: Thread %d still has value %d\n",
            omp_get_thread_num(), thread_private_var);
    }
    return 0;
}
```

threadprivate

First parallel region: Thread 0 has value 0

First parallel region: Thread 4 has value 4

First parallel region: Thread 1 has value 1

First parallel region: Thread 5 has value 5

First parallel region: Thread 3 has value 3

First parallel region: Thread 2 has value 2

First parallel region: Thread 6 has value 6

First parallel region: Thread 7 has value 7

Second parallel region: Thread 3 still has value 3

Second parallel region: Thread 1 still has value 1

Second parallel region: Thread 6 still has value 6

Second parallel region: Thread 2 still has value 2

Second parallel region: Thread 0 still has value 0

Second parallel region: Thread 4 still has value 4

Second parallel region: Thread 5 still has value 5

Second parallel region: Thread 7 still has value 7

Try the code...

```
#include <stdio.h>
#include <omp.h>
#define N 1000

int main() {
    int array[N];
    int sum = 0;

    for(int i = 0; i < N; ++i) {
        array[i] = 1;
    }

    #pragma omp parallel shared(array) private(sum)
    {
        int local_sum = 0;
        #pragma omp for
        for(int i = 0; i < N; ++i) {
            local_sum += array[i];
        }
        #pragma omp atomic
        sum += local_sum;
    }

    printf("Sum: %d\n", sum);
    return 0;
}
```

Expected value: 1000

activities > variables > C sharedSum.c > main()

```
1  #include <stdio.h>
2  #include <omp.h>
3  #define N 1000
4
5  int main()
6  {
7      int array[N];
8      int sum = 0;
9      for(int i=0; i<N; ++i){
10         array[i] = 1;
11     }
12     #pragma omp parallel shared(array, sum) //sum shared
13     {
14         int local_sum =0;
15
16
17         #pragma omp for
18         for(int i=0; i<N; ++i){
19             local_sum += array[i];
20         }
21
22
23         #pragma omp atomic
24         sum += local_sum;
25     }
26     printf("Sum: %d\n", sum);
27     return 0;
28 }
29
```

Make sum shared...

private(sum) gave each thread its own uninitialized copy, the global sum was never updated.

Making it shared let's all threads write back to the same variable.


```
1  #include <stdio.h>
2  #include <omp.h>
3
4
5  #define N 1000
6
7  int main()
8  {
9      int array[N];
10     int sum = 0;
11     for(int i=0; i<N; ++i){
12         array[i] = 1;
13     }
14     #pragma omp parallel for reduction(+:sum)
15     for(int i=0; i<N; ++i){
16         sum += array[i];
17     }
18     printf("Sum: %d\n", sum);
19     return 0;
20 }
21
22
23
24
```

Using reduction..

What does reduction do internally?

Synchronization

Synchronization in OpenMP is the process of coordinating thread execution to ensure data consistency and prevent race conditions. It establishes an order of operations when multiple threads access shared resources

- Critical
- Atomic
- Barrier
- Ordered
- Master
- Single

Master

- Only the master thread executes the code block.
- Used when only the main thread should do a task like managing data.
- Only thread 0

Master

```
#include <omp.h>
#include <stdio.h>

// Only the master thread executes this section
int main() {
    #pragma omp parallel num_threads(4)
    {
        // All threads execute this
        printf("Thread %d is working\n", omp_get_thread_num());

        #pragma omp master
        {
            printf("This is only executed by master thread (%d)\n",
                omp_get_thread_num());
        }

        // No implicit barrier after master
        printf("Thread %d continues\n", omp_get_thread_num());
    }
    return 0;
}
```

Thread 3 is working
Thread 3 continues
Thread 0 is working
This is only executed by master thread (0)
Thread 2 is working
Thread 2 continues
Thread 1 is working
Thread 1 continues
Thread 0 continues

Single

- Only one thread executes the block, but it can be any thread.
- Other threads wait unless `nowait` is use. This will avoids repeating the same task by all threads.
- Only one thread.

Single

```
#include <omp.h>
#include <stdio.h>

// Only one thread (not necessarily master) executes this section
int main() {
    #pragma omp parallel num_threads(4)
    {
        printf("Thread %d is working\n", omp_get_thread_num());

        #pragma omp single
        {
            printf("This section is executed by thread %d\n",
                omp_get_thread_num());
        }
        // Implicit barrier here

        printf("Thread %d continues after single\n", omp_get_thread_num());
    }
    return 0;
}
```

Thread 0 is working
This section is executed by thread 0
Thread 1 is working
Thread 3 is working
Thread 2 is working
Thread 1 continues after single
Thread 0 continues after single
Thread 3 continues after single
Thread 2 continues after single

Critical

Used when only one thread at a time should execute a block of a code.

Prevents multiple threads from changing shared data at the same time. Without critical, two threads may update the same variable together and produce wrong results.

Critical

```
#include <omp.h>
#include <stdio.h>

// Ensures only one thread executes the code block at a time
int main() {
    int sum = 0;

    #pragma omp parallel num_threads(4)
    {
        int local_sum = 0;
        for(int i = 0; i < 1000; i++) {
            local_sum++;
        }

        #pragma omp critical
        {
            sum += local_sum;
            printf("Thread %d added its local sum\n", omp_get_thread_num());
        }
    }

    printf("Final sum: %d\n", sum);
    return 0;
}
```

Thread 1 added its local sum
Thread 0 added its local sum
Thread 2 added its local sum
Thread 3 added its local sum
Final sum: 4000

Barrier

- Makes all threads wait until every thread reaches the same point.
- No thread can continue until all others arrive. This ensures all threads finish one step before moving to the next step.

Barrier

```
#include <omp.h>
#include <stdio.h>
#include <unistd.h>

// Makes threads wait until all threads reach the barrier point

int main() {
    #pragma omp parallel num_threads(4)
    {
        printf("Thread %d starting phase 1\n", omp_get_thread_num());
        sleep(omp_get_thread_num()); // Simulate different work loads

        #pragma omp barrier

        printf("Thread %d starting phase 2\n", omp_get_thread_num());
    }
    return 0;
}
```

Thread 2 starting phase 1
Thread 0 starting phase 1
Thread 1 starting phase 1
Thread 3 starting phase 1

Thread 0 starting phase 2
Thread 1 starting phase 2
Thread 2 starting phase 2
Thread 3 starting phase 2

Atomic

- Used for very small operations on shared variables safely.
- Prevents the race condition during simple updates.
- Usually faster than critical but only used for simple operations.

```
#include <omp.h>
#include <stdio.h>

// Provides lightweight synchronization for single operations
int main() {
    int counter = 0;

    #pragma omp parallel num_threads(4)
    {
        for(int i = 0; i < 1000; i++) {
            #pragma omp atomic
            counter++;
        }
    }

    printf("Final counter value: %d\n", counter);
    return 0;
}
```

Output: Final counter value: 4000

Ordered

- Used when you want a part of a parallel loop to run in the original sequential order.
- Even Though threads run in parallel, the ordered section executes one by one in loop order.

Try the code ..

```
Activities > Synch > C tryCode.c > ...
1  #include <stdio.h>
2  #include <omp.h>
3  #define N 1000
4
5  int main() {
6      int array[N];
7      int sum = 0;
8      int count = 0;
9
10     for(int i = 0; i < N; ++i) {
11         array[i] = 1;
12     }
13
14     #pragma omp parallel shared(array, sum, count)
15     {
16         int local_sum = 0;
17         int local_count = 0;
18         #pragma omp for
19         for(int i = 0; i < N; ++i) {
20             local_sum += array[i];
21             local_count++;
22         }
23         #pragma omp atomic          // Problem: only protects ONE statement
24         sum += local_sum;
25         count += local_count;      // This line is NOT protected!
26     }
27
28     printf("Sum: %d, Count: %d\n", sum, count);
29     return 0;
30 }
31
```

Atomic will only protects one statement.

Count is being updated without protection.

Solution: Use
critical for
multiple lines.

```
int main() {
    int array[N];
    int sum = 0;
    int count = 0;

    for(int i = 0; i < N; ++i) {
        array[i] = 1;
    }

    #pragma omp parallel shared(array, sum, count)
    {
        int local_sum = 0;
        int local_count = 0;
        #pragma omp for
        for(int i = 0; i < N; ++i) {
            local_sum += array[i];
            local_count++;
        }
        #pragma omp critical // Update to critical
        sum += local_sum;
        count += local_count;
    }

    printf("Sum: %d, Count: %d\n", sum, count);
    return 0;
}
```

Work Distribution

Divides work among multiple threads so that they can run in parallel.

1. Static schedule - `schedule(static, 2)`
 - a. Iterations preassigned in fixed chunks.
 - b. Low overhead, good for uniform work.
2. Dynamic schedule - `schedule(dynamic, 2)`
 - a. Threads grab chunks on demand. Higher overhead, better for variable workloads.
3. Sections
 - a. Different code blocks run in parallel.
 - b. Good for distinct independent tasks.
4. Tasks
 - a. One thread creates tasks, others execute them.
 - b. Flexible for irregular workloads.

Static Schedule.

Loop iterations are divided among threads before execution starts. Each thread gets a fixed portion of work.

```
#include <omp.h>
#include <stdio.h>
```

```
int main() {
    const int size = 16;

    #pragma omp parallel for schedule(static, 2)
    for(int i = 0; i < size; i++) {
        printf("Thread %d processes iteration %d\n",
            omp_get_thread_num(), i);
    }
    return 0;
}
```

With 4 threads, 16 iterations, chunk = 2

Thread 0 gets {0, 1} , {8,9}

Thread 1 gets {2, 3}, {10,11} deterministic

Thread 1 processes iteration 2

Thread 1 processes iteration 3

Thread 3 processes iteration 6

Thread 3 processes iteration 7

Thread 6 processes iteration 12

Thread 6 processes iteration 13

Thread 7 processes iteration 14

Thread 7 processes iteration 15

Thread 0 processes iteration 0

Thread 0 processes iteration 1

Thread 5 processes iteration 10

Thread 5 processes iteration 11

Thread 4 processes iteration 8

Thread 4 processes iteration 9

Thread 2 processes iteration 4

Thread 2 processes iteration 5

Dynamic Schedule.

Work is assigned during execution. Threads will take new work whenever they finish previous work.

```
#include <omp.h>
#include <stdio.h>

int main() {
    const int size = 16;

    #pragma omp parallel for schedule(static, 2)
    for(int i = 0; i < size; i++) {
        printf("Thread %d processes iteration %d\n",
            omp_get_thread_num(), i);
    }
    return 0;
}
```

With 4 threads, 16 iterations, chunk = 2
Thread 0 gets {0, 1} , {8,9}
Thread 1 gets {2, 3}, {10,11} deterministic

```
#include <omp.h>
#include <stdio.h>
```

```
int main() {
    #pragma omp parallel
    {
        #pragma omp sections
        {
            #pragma omp section
            {
                printf("Section 1 executed by thread %d\n",
                    omp_get_thread_num());
            }

            #pragma omp section
            {
                printf("Section 2 executed by thread %d\n",
                    omp_get_thread_num());
            }

            #pragma omp section
            {
                printf("Section 3 executed by thread %d\n",
                    omp_get_thread_num());
            }
        }
    }
    return 0;
}
```

Sections

Divides completely different tasks among threads. Each section block is executed by one thread.

If we have 3 sections and 4 threads, then one thread sits idle.

Tasks

Creates small independent jobs that threads execute dynamically. Tasks are flexible.

```
#include <omp.h>
#include <stdio.h>

void process_item(int i) {
    printf("Task %d processed by thread %d\n", i, omp_get_thread_num());
}

int main() {
    #pragma omp parallel
    {
        #pragma omp single // Only one thread creates tasks
        {
            for(int i = 0; i < 5; i++) {
                #pragma omp task
                process_item(i);
            }
        }
    }
    return 0;
}
```

Ensure only one thread creates the tasks, and rest help execute them. Good for graphs/ trees

```

#include <stdio.h>
#include <omp.h>
#include <stdlib.h>

#define N 4
int main()
{
    int i, j, k;
    int A[N][N], B[N][N], C[N][N];

    // initialize matrices A and B
    for(i=0; i<N; i++){
        for(j=0; j<N; j++){
            A[i][j] = i+j;
            B[i][j] = i-j;
            C[i][j] = 0; //initialize results matrix C to zero
        }
    }

    #pragma omp parallel for private(i, j, k) shared(A, B, C)
    for(i=0; i<N; i++){
        for(j=0; j<N; j++){
            for(k=0; k<N; k++){
                C[i][j] += A[i][k] * B[k][j];
            }
        }
    }

    printf("Resulting matrix C has been computed \n");
    return 0;
}

```

Matrix Multiplication.

Only outer loop parallelized.

Add collapse(2)

```
#include <stdio.h>
#include <omp.h>
#include <stdlib.h>

#define N 4
int main()
{
    int i, j, k;
    int A[N][N], B[N][N], C[N][N];

    // initialize matrices A and V
    for(i =0; i<N; i++){
        for(j=0; j<N; j++){
            A[i][j] = i+j;
            B[i][j] = i-j;
            C[i][j] = 0; //initialize results matrix C to zero
        }
    }

    #pragma omp parallel for collapse(2) private(i, j, k) shared(A, B, C)
    for(i =0; i<N; i++){
        for(j=0; j<N; j++){
            for(k=0; k<N; k++){
                C[i][j] += A[i][k] * B[k][j];
            }
        }
    }

    printf("Resulting matrix C has been computed \n");
    return 0;
}
```


Activity

<https://acesse.one/lvq7uhw>

Use the above code and fill in the gaps

Task 1 — Fill in the `#pragma` line inside `compute_pi_parallel()`.

Task 2 — Add timing around both function calls in `main()` using `omp_get_wtime()`.

Task 3 — Change `NUM_STEPS` to 100, then 100000, then 10000000.

Task 4 — What happens if you remove `private(x)` from the `pragma`?